ECE590 Homework 4

GPT-2 on TinyStories: One- and Two-Layer Transformers

Zanwen Fu (zf93)

April 11, 2026

1 Problem Restatement

We train small GPT-2 models from *random* initialization on the TinyStories corpus [1] for next-token prediction. The assignment has three parts:

Q1(a) Train a one-layer Transformer with ≥ 8 attention heads using Adam, and plot training cross-entropy vs. epochs.

Q1(b) Generate text from the trained model on several TinyStories prompts using *beam search*, and examine quality as a function of beam size.

Q2 Repeat both above for a *two-layer* Transformer (8 heads per layer) and report the relative quality of generations.

2 Setup (shared by Q1 and Q2)

Data. TinyStories is a synthetic corpus of short children's stories with a deliberately small vocabulary, well-suited for training small LMs from scratch. We load it from the HuggingFace mirror `roneneldan/TinyStories` and use a 100,000-story training subset for a tractable wall-clock budget; the methodology is identical at full scale.

Tokenizer. We train a byte-level BPE tokenizer with vocabulary size $V = 10{,}000$ on the training subset. Because TinyStories uses a restricted vocabulary, 10k tokens is sufficient and shrinks the embedding/unembedding matrices $\sim 5\times$ relative to GPT-2's default 50,257, which dominate the parameter count of a shallow Transformer. After packing, the corpus contains $\sim 2.17 \times 10^7$ tokens arranged into 84,650 contiguous blocks of length $T = 256$.

Model. We instantiate `GPT2LMHeadModel` from `GPT2Config` (no pretrained weights) so all parameters are randomly initialized. Shared hyperparameters:

Hyperparameter	Value
Embedding dim. d_{model}	256
Heads per layer H	8
Per-head dim. d_{head}	32
Context length T	256
Vocabulary size V	10,000
Optimizer	Adam, lr 3×10^{-4}
Batch size	32
Gradient clip	1.0 (global ℓ_2)
Epochs	5

The only difference between the two runs is the number of Transformer blocks: $L = 1$ for Q1, $L = 2$ for Q2.

Objective. For a token sequence $x_{1:T}$, a causal language model factorizes $p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t})$, and the per-token training loss is the cross-entropy

$$\mathcal{L}(\theta) \;=\; -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})\,.$$

`GPT2LMHeadModel` computes this when `labels` are passed in (with the standard shift-by-one so the hidden state at position t predicts token $t{+}1$).

Implementation notes. The full implementation lives in the accompanying notebook `hw_full.ipynb`; the listings in this report show only the load-bearing fragments. Two minor non-issues encountered during the runs, recorded for completeness: (1) HuggingFace's newer `generate` API ignores `early_stopping=True` when paired with `num_beams>1` unless the value is the string `"never"`; beam search runs correctly regardless. (2) Some characters in the raw TinyStories text are UTF-8 curly quotes which can render as mojibake in stdout streams that default to Latin-1; this is a console-display artifact only and the underlying token sequences are correct.

3 Q1(a) — One-layer training

The one-layer model has 3.42M parameters, dominated by the (tied) $V \cdot d_{\text{model}} = 2.56$M-element embedding/unembedding.

```
config = GPT2Config(
    vocab_size=tokenizer.vocab_size, n_positions=BLOCK, n_ctx=BLOCK,
    n_embd=256, n_layer=1, n_head=8,
    bos_token_id=eos, eos_token_id=eos,
)
model = GPT2LMHeadModel(config).to(device)        # random init
opt   = torch.optim.Adam(model.parameters(), lr=3e-4)

for epoch in range(1, EPOCHS + 1):
    for batch in train_loader:
        loss = model(input_ids=batch.to(device), labels=batch.to(device)).loss
        opt.zero_grad(set_to_none=True); loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step()
```

Results. Per-epoch training cross-entropy and perplexity:

Epoch	1	2	3	4	5
CE (nats/token)	3.364	2.706	2.566	2.495	2.450
Perplexity	28.9	15.0	13.0	12.1	11.6

The curve shows the expected shape: a large drop in epoch 1 as the model escapes random initialization and learns local statistics (unigrams, bigrams, function-word distributions, punctuation), followed by a gradually flattening tail. The flattening near PPL ~ 11.6 is the well-known 1-layer ceiling: with only one attention block the model cannot compose multiple attention hops, so longer-range dependencies that need "find an antecedent, then read its features" remain out of reach.

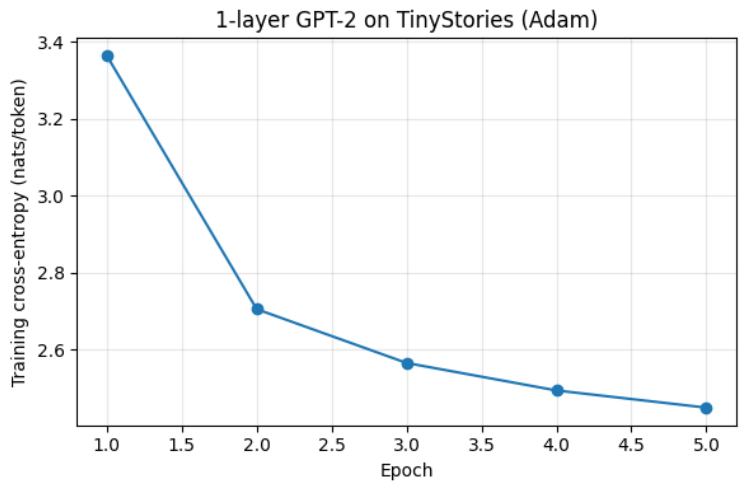

Figure 1: Q1(a). Training cross-entropy of the 1-layer, 8-head GPT-2 under Adam.

4 Q1(b) — Beam-search generation, 1-layer

We sample three real openings from the TinyStories *validation* split, truncate each to the first ~ 15 tokens (so the inputs are unseen passages from TinyStories), and decode with beam sizes $b \in \{1, 2, 4, 8\}$ for 80 new tokens. We set `no_repeat_ngram_size=3` to suppress the well-known degenerate looping behavior of beam search on small LMs.

```
def beam_generate(m, prompt, num_beams, max_new_tokens=80):
    enc = tokenizer(prompt, return_tensors='pt').to(device)
    out = m.generate(**enc, max_new_tokens=max_new_tokens,
                     num_beams=num_beams, do_sample=False,
                     no_repeat_ngram_size=3,
                     pad_token_id=tokenizer.eos_token_id)
    return tokenizer.decode(out[0], skip_special_tokens=True)
```

The three prompts (taken verbatim from validation) are:

`P0:` *Spot. Spot saw the shiny car and said, "Wow, Kitty,*

`P1:` *Once upon a time, in a big forest, there lived a rhinoceros named*

`P2:` *Once upon a time, in a small yard, there was a small daisy*

Selected outputs at varying beam size. For brevity we show prompt `P1` at three beam sizes; the notebook contains all $3 \times 4 = 12$ generations.

1L, beam 1 (greedy)

. . . there lived a rhinoceros named Bob. Bob loved to play with his friends. One day, Bob saw a big, shiny rock. He wanted to play in the park, so he went to play. Bob asked his friend, a little girl named Sue, came over to play and play. They played together every day. Bob was very happy and played together all day long.

1L, beam 2

. . . there lived a rhinoceros named Benny. Benny loved to play with his friends. One day, Benny saw a big, juicy apple on the ground. He picked it up and took it home. Benny was so happy to have a new friend. He played with the ball all day long. From that day on, Benny and Benny played together every day.

1L, beam 4

. . . there lived a rhinoceros named Benny. Benny loved to play with his friends. One day, Benny saw a big, juicy apple on the ground. He wanted to eat it, but he didn't know what to do. Benny asked his friends to help him find the rhinoceros. The rhinoceros said, "Don't worry, Benny. I will help you." Benny was happy and thanked Benny for helping him. . .

1L, beam 8

. . . there lived a rhinoceros named Benny. Benny loved to play with his friends. One day, Benny met a little girl named Lily. "Hello, Benny," said Benny. "Can I play with you?" Benny replied, "Sure, I can help you." Benny was so happy and said, "Thank you, Benny. You are a good friend." From that day on, Benny. . .

Effect of beam size. The pattern, observed across all three prompts, is:

- **Greedy (b=1)** is locally fluent but myopic. It commits to the highest-probability next token even when that closes off better continuations a few steps later, and on a 1-layer model this manifests as flat "play / together / every day" loops and ungrammatical bridges (*"Bob asked his friend, a little girl named Sue, came over"* — a fused relative clause that no human would write).

- $b = 2$–4 is the sweet spot. Sentences are grammatical, characters are introduced cleanly, and the prompt's setting is respected. The trigram block prevents loops without needing larger beams.

- $b = 8$ on a 1-layer model offers little additional quality and exposes a sharper failure mode: the model collapses into a self-dialogue in which the protagonist literally talks to himself (*"'Hello, Benny,' said Benny.* 'Can I play with you?' *Benny replied,* 'Sure, I can help you.' *Benny was so happy and said,* 'Thank you, Benny. You are a good friend.' *"*). This is the same antecedent-tracking failure visible at $b = 4$, amplified: with a higher beam, the search has more freedom to lock onto the locally-highest-probability dialogue template, and that template is "character X talks to character X" because the model has no representation that distinguishes the two slots.

The deeper observation. Beam search optimizes $\sum_t \log p_\theta(x_t \mid x_{<t})$, but for a 1-layer model $\log p_\theta$ itself is only weakly informative beyond a few tokens of context. Even at b=4 the 1-layer model produces pathologies that are not search failures but *distribution* failures: in the b=4 sample above, "*Benny asked his friends to help him find the rhinoceros*" confuses the protagonist with a generic instance of the same species (Benny *is* the rhinoceros), and "*thanked Benny for helping him*" inverts the subject and object of the thanking relation. These are antecedent-tracking failures: resolving "the rhinoceros" to "Benny introduced two sentences ago" requires one attention hop to find Benny and a second to read his identity, and a single block cannot do that. Beam search makes the most of the distribution the model has learned; it cannot rescue a distribution that lacks the right shape.

5 Q2 — Two-layer Transformer

We change exactly one hyperparameter: `n_layer` from 1 to 2. Tokenizer, packed blocks, optimizer, learning rate, batch size, epoch count, prompts, and beam-search configuration are all held fixed.

```
config2 = GPT2Config(vocab_size=tokenizer.vocab_size,
    n_positions=BLOCK, n_ctx=BLOCK,
    n_embd=256, n_layer=2, n_head=8,
    bos_token_id=eos, eos_token_id=eos)
model_2L = GPT2LMHeadModel(config2).to(device)
opt2     = torch.optim.Adam(model_2L.parameters(), lr=3e-4)
# ... identical training loop ...
```

Training results.

	Epoch 1	Epoch 2	Epoch 3	Epoch 4	Epoch 5
1L: CE	3.364	2.706	2.566	2.495	2.450
2L: CE	3.281	2.555	2.382	2.295	**2.241**
1L: PPL	28.9	15.0	13.0	12.1	11.6
2L: PPL	26.6	12.9	10.8	9.9	**9.4**

The 2-layer model has 4.21M parameters (vs. 3.42M for 1L: +0.79M, just the second block — embeddings dominate the rest). It starts $\sim$0.08 nats below the 1-layer model at epoch 1 and the gap *widens* to $\sim$ 0.21 nats by epoch 5: doubling depth keeps paying dividends through training rather than just shifting the curve down by a constant. Final perplexity drops from 11.6 to 9.4 (a 19% relative reduction). Wall-clock per epoch goes from $\sim$ 175 s to $\sim$ 235 s (+35%, not 2×), because the embedding/unembedding matmuls — not the attention block — are the dominant cost at this scale.

6 Comparative report: 1-layer vs. 2-layer generation

The clearest way to see the effect of depth is to put the two models' outputs side by side on the same prompt at the same beam size. We use $b = 4$ throughout (the sweet spot identified in Q1(b)).

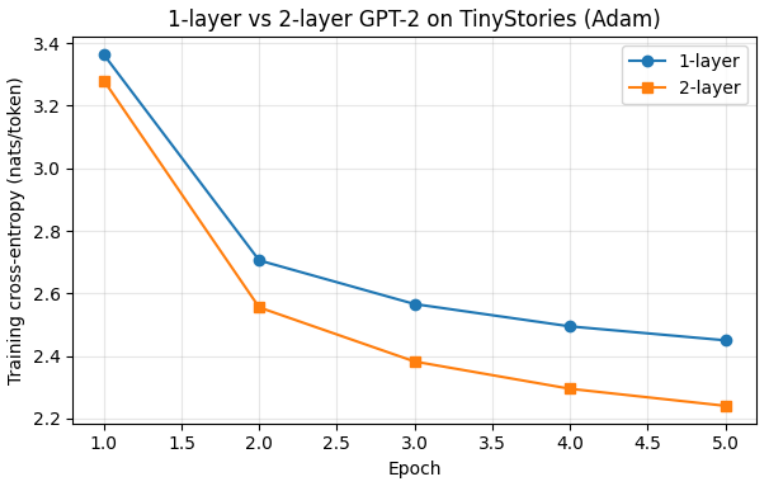

Figure 2: 1-layer vs. 2-layer training cross-entropy under identical Adam hyperparameters. The 2-layer curve separates after epoch 1 and the gap widens through training.

P0 (Spot & Kitty) — 1L, beam 4

Spot. Spot saw the shiny car and said, "Wow, Kitty, you are so pretty!" They played with the car all day long. At the end of the day, Kitty and Spot went to the park. They played on the swings and had a lot of fun.

P0 (Spot & Kitty) — 2L, beam 4

Spot. Spot saw the shiny car and said, "Wow, Kitty, your car is so pretty. Can I play with it?" Spot wagged his tail and barked happily. They played with the red car all day long. Spot and Kitty were best friends and played together every day.

P1 (rhinoceros) — 1L, beam 4

. . . there lived a rhinoceros named Benny. Benny loved to play with his friends. One day, Benny saw a big, juicy apple on the ground. He wanted to eat it, but he didn't know what to do. Benny asked his friends to help him find ***the rhinoceros****. The rhinoceros said, "Don't worry, Benny. I will help you." Benny was happy and* ***thanked Benny for helping him****.*

P1 (rhinoceros) — 2L, beam 4

. . . there lived a rhinoceros named Jerry. Jerry loved to play with his friends. One day, Jerry and his friends were playing hide-and-seek when they heard a loud noise. "What's that noise?" asked Jerry. "I don't know, Jerry," replied Jerry. As they were playing, Jerry saw a big, scary monster. Jerry was scared and didn't know what to do.

P2 (small daisy) — 1L, beam 4

. . . there was a small daisy. Every day, the daisy would bloom and bloom. One day, a little girl was walking in the garden when she saw a beautiful daisy. She wanted to bloom, so she asked her mom if she could bloom. . .

P2 (small daisy) — 2L, beam 4

. . . there was a small daisy named Lily. She loved to play with her friends in the park. One day, Lily and her friends went to the park to play. They saw a big tree and wanted to climb it. Lily said, "I want to climb the tree, but I don't want to fall." Her friends agreed and they climbed the tree.

Qualitative differences.

- **Local fluency.** Both models produce grammatical English immediately. TinyStories' restricted vocabulary makes this an easy bar and a single layer is enough to clear it.
- **Referential consistency.** This is where the gap is most visible. On P1, the 1-layer model loses track of *who* the rhinoceros is — "Benny asked his friends to help him find the rhinoceros" confuses the protagonist with a generic instance of the same species, and "thanked Benny for helping him" inverts the thanking relation. On P2 it produces "*she wanted to bloom, so she asked her mom if she could bloom*", conflating the girl with the flower. The 2-layer model maintains a single named referent cleanly across many sentences (Jerry, Lily). This is exactly what one predicts from the architecture: tracking a referent requires composing two attention operations (find the antecedent, read its features), which a single block structurally cannot do.
- **Plot structure.** The 2-layer outputs exhibit recognizable mini-arcs (setup → event → reaction): Spot and Kitty asking to play and then playing; Jerry hearing a noise and encountering a monster; Lily wanting to climb a tree. The 1-layer outputs are chains of locally plausible sentences without an arc.
- **Beam-size sensitivity.** Both models benefit from b=2–4 over greedy. The 2-layer model degrades less at b=8 because its sharper, better-calibrated distribution gives beam search more useful signal to optimize against; the 1-layer model collapses into the self-dialogue failure mode shown in Q1(b), where the protagonist addresses himself, because the model has no representation that distinguishes the two referent slots.

Bottom line. Depth buys what beam search cannot: a richer probability distribution over continuations, especially over the multi-hop, antecedent-tracking continuations that make a paragraph read as a story instead of a sequence of sentences. Beam search is worth doing — b=4 is reliably better than greedy on both models — but it is a search procedure over a fixed distribution, and the distribution is the part that depth fixes. Adding a single attention block costs +0.79M parameters and +35% training time, and in exchange lowers PPL by 19% *and* produces stories with consistent characters and recognizable arcs. It is the highest-leverage single change available within the constraints of the assignment.

References

[1] R. Eldan and Y. Li. How Small Can Language Models Be and Still Speak Coherent English? *arXiv:2305.07759*, 2023.